Grile POO 2014

1. Identificați afirmațiile corecte din următorul set:

A. O clasă imutabila nu permite existența de metode de tip “setter” publice (definite cu identificatorul de acces public)

B. Spre deosebire de variabile și metode, clasele nu pot fi definite private.

C. Interfețele pot extinde alte interfețe.

D. După instantierea unui vector (de exemplu: new Object[10]), indiferent de tipul elementelor, toate au valoarea null.Optiuni:

- A,B,C
- **C**
- B,C
- B, D

A - fals, nu exista aceasta retrictie, B - clasele interne pot fi private, C - adevarata, interfetele pot extinde alte interfete, D - falsă, in cazul Object se inițializează cu null, dar pentru boolean cu false, int/short/bye cu 0 iar float/double cu 0.0. La test varinta corectă cu C aparea cu C, D → a fost “anulata” intrebarea - considerată corectă pentru toți.

2. Care dintre următoarele metode suprascrie metoda : public void suprascrie (int a, String b) {}

- public void suprascrie (String b, int a) {}
- public String suprascrie (int a, String b) {}
- **public void suprascrie (int integer, String string) {}**
- public void suprascrie (int a, String b) throws IOException {}

R: Întrebarea verifică cunoașterea regulilor pentru supracriere:

- aceeași listă de argumente
- același tip de return sau un subtip al acestuia
- nu pot avea un modificator de acces mai restrictiv
- pot arunca doar aceleași excepții, excepții derivate din acestea, sau excepții unchecked.

Doar varianta public void suprascrie (int integer, String string) {} respectă aceste reguli.

3. Ce se afișează la execuția următorului cod:

```
class MyClass {
   private int id;

   public MyClass(int id) {
      this.id = id;
   }
```

```
    public int getId() {
       return id;
    }
    @Override
    public boolean equals(MyClass obj) {
        return obj.id == id;
    }

    public static void main(String []args){
       MyClass class1 = new MyClass(3);
       MyClass class2 = new MyClass(new Integer(3));

       if (class1.equals(class2))
          System.out.println("Obiectele sunt egale");
       else
             System.out.println("Obiectele difera");
     }
}
```

- Se generează eroare de compilare la linia MyClass class2 = new MyClass(new Integer(3));
- Se afișează “Obiectele sunt egale”
- Se afișează “Obiectele diferă”
- **Se generează o altă eroare de compilare față de cea menționată la celalalta varianta de răspuns**

R: Se generează eroare la @Override pentru că signatura funcției equals din Object primește parametru Object, ca atare equals-ul definit nu este o suprascriere, ci o supraîncărcare.

4. Care dintre următoarele afirmații nu definește încapsularea?

- gruparea atributelor și operațiilor caracteristice unui obiect
- modificarea stării interne a unui obiect strict prin intermediul operatiilor acestuia
- **posibilitatea implementării interfețelor**
- ascunderea modului de funcționare a unui obiect

R: Scopul întrebării este verificarea cunoașterii conceptului de încapsulare. Posibilitatea implementării interfețelor nu are legătură cu conceptul OOP de încapsulare. Restul afirmațiilor definesc caracteristici/avantaje ale încapsulării.

5. Ce se afișează la execuția următorului cod:

```
class A {
        private int x = 5;
```

```
        private void hidden() {
            System.out.println(x);
        }

        public void show_hidden() {
            hidden();
        }
    }
    class B extends A {
        public int x = 10;

        public void hidden() {
            System.out.println(x);
        }
    }
    public class Main {
        public static void main(String[] args) {
            B b = new B();
            b.show_hidden();
        }
    }
```

- eroare la compilare
- **5**
- 10
- 0

R: La execuție se afișează 5 pentru că variabila x din clasa A este private, altfel ar fi fost 10.

6. Fie următorul program:

```
class C {}

class D extends C {}

class A {
    static void foo (C c) { … }
    static void foo (D d) { … }
}

class B extends A {
    static void foo (C c) { … }
    static void foo (D d) { … }
}
```

```
…(in main)
A a = new B();
C c = new D();
```

Care metodă va fi apelată dacă se rulează a.foo©?

- **foo (C) din A**
- foo (C) din B
- foo (D) din A
- foo (D) din B

R: Exercițiu similar celui din laboratorul de recapitulare, însă pentru metode statice. Scopul acestui exercițiu este verificarea cunoașterii conceptului că metodele statice nu pot fi suprascrise.

7. Ce se afișează la execuția următorului cod:

```
public class Main {
   public static void main(String []args) {
      B obj = new B();
      System.out.println(obj.getValue());
   }
}
class A {
   public int x;

   public A() {
      x = 10;
   }

   public int getValue() {
      return x;
   }
}
class B extends A {
   public int x;

   public B() {
      x = 20;
   }
}
```

- **10**
- 20
- Se generează eroare la compilare

- Niciuna din variantele de mai sus

R: Din moment ce metoda getValue nu este suprascrisă în B, atunci se apelează cea din A, pentru care este vizibil x-ul din A, care are valoarea 10.

8. Care afirmație este adevărată în contextul limbajului Java?

- O clasă poate implementa oricâte interfețe și poate moșteni oricâte clase (abstracte sau concrete)
- O clasă poate implementa o singură interfață și poate moșteni oricâte clase (abstracte sau concrete)
- **O clasă poate implementa oricâte interfețe și poate moșteni o singură clasă (abstractă sau concretă)**
- O clasă poate implementa oricâte interfețe și poate moșteni oricâte clase abstracte și o singură clasă concretă

R: Întrebarea verifică regula de bază legată de moștenire în Java: poate fi moștenită o singură clasă dar implementate oricâte interfețe.

9. Care variantă definește cel mai bine legătura între interfețe și clase?

- atât clasele, cât și interfețele definesc modul în care un obiect execută o operație
- **interfețele precizează operațiile expuse de un obiect, în timp ce clasele modul în care acesta le execută**
- nici clasele, nici interfețele nu precizează modul în care un obiect execută o operație
- o clasă nu definește implicit interfața instanțelor sale

R: Întrebarea verifică cunoștiințele legate de scopul și utilizarea interfețelor și cu ce diferă față de cel al claselor. Raspunsul corect este că interfețele expun operațiile unui obiect (un contract a ceea ce face/oferă un obiect) iar clasele modul în care le execută.

10. Ce va afișa următorul program:

```
class Outer {
        static class Inner {
                public static void main(String[] args){
                        System.out.print("1");
                }
        }

        public static void main(String[] args) {
                Outer.Inner.main(null);
                System.out.print("2");
        }
}
```

- **12**

- eroare compilare, nu putem apela explicit main din Inner
- 2
- eroare compilare, nu putem avea metoda main intr-o clasa internă

R: Declarația și apelul metodei main este corect. Clasele interne nu pot fi executate direct, însă nimic nu restricționează apelul acelei metode, din alta parte în program. Se apelează main și se afișează “1”, apoi se afișează “2”.

11. Alegeți codul corect pentru instanțierea unei clase interne anonime de tipul Food (clasă abstractă ce conține metoda public abstract String eat();)

- Food f = new Food()
- System.out.println(new Food(public String eat(){return “eat pizza”;}));
- System.out.println(new Food{String eat(){return “eat pizza”;}});
- **System.out.println(new Food(){public String eat(){return ””;}});**

R: Scopul acestui exercițiu este verificarea cunoaștiințelor legate de clasele interne anonime, mai precis felul în care acestea sunt create, și ca pot fi create în mod convenabil și acolo unde se dau parametru pentru o altă metodă.

12. Care afirmație despre clasele interne statice este adevărată?

- trebuie să avem o referință la instanța clasei exterioare pentru a o putea instanția
- **din clasa internă statică nu avem acces la membrii non-statici ai clasei exterioare**
- variabilele și metodele ei trebuie să fie statice
- o clasă internă statică nu poate avea specificatorul de acces private

R: Scopul acestui exercițiu este verificarea cunoștiințelor legate de clasele interne statice, mai precis conceptul de bază, că nu avem nevoie de referință către instanța clasei exterioare. Deci nu avem acces la membrii non statici ai acesteia, pentru ca nu avem o instanță în care să “existe” aceștia.

13. Ce puteți spune despre următoarea funcție?

```
public String f() {
   String s=”1”;
   try {
      throw new Exception();
   } catch (Exception e) {
      try {
         try {
               throw new Exception();
           } catch (Exception e1) {
               s += "2";
           }
                  throw new Exception();
         } catch (Exception e2) {
```

```
            s += "3";
            return s;
          }
          finally {
            s += "5";
          }
        }
      finally {
        s += "6";
      }
    }
```

- generează 1 eroare de compilare
- generează 2 erori de compilare
- **la execuție întoarce 123**
- la execuție întoarce 12356

R: Răspunsul corect este șirul “123” deoarece, deși se execută instrucțiunile din finally, pe stivă s-a pus deja ca rezultat returnat versiunea “123” a șirului. Obiectele String sunt immutable, deci modificările aduse în finally au rezultat într-un alt obiect string, nemodificându-se șirul pus să fie returnat. Dacă era un return în ultimul finally atunci s-ar fi întors 12356. Conform specificației Java, blocul finally se execută înainte de transferul controlului (return-ul efectiv), dar expresia returnată (în cazul nostru șirul) va fi cea de dinainte de întrepunerea finally-ului.

14. Ce va afișa următoarea secvență de cod?

```
public int f(int n) {
  int s = 0;
  for (int i =0; i < n; i++) {
    s += f(1);
  }
  return s;
}
...
try {
  System.out.println(f(2));
} catch (Throwable t) {
  System.out.println("am prins ceva!");
}
```

- 2
- eroare de compilare, obiectele Throwable nu pot fi prinse
- va intra într-o buclă infinită
- **am prins ceva!**

R: Secvența de cod conține un apel recursiv infinit, ce generează o excepție StackOferflow, prinsă în blocul catch. La laboratorul de excepții a fost un exercițiu care să genereze aruncarea unei astfel de excepții.

15. Ce va afișa următoarea secvență de cod?

```
public class Main {
   public static int f(int a, int b) {
      try {
         System.out.println(a/b);
      } catch (Exception ex) {
         return 1;
      }
      finally {
         return 2;
      }
   }

   public static void main(String []args) {
      int result = f(10, 0);
      System.out.println(result);
   }
}
```

- 1
- **2**
- eroare la compilare
- eroare la rulare, excepția generată nu poate fi prinsă

R: Scopul exercițiului este verificarea înțelegerii flow-ului try-catch-finally. Secvența de cod generează o excepție la împărțirea cu 0, și atunci se intră în catch și apoi în finally, return-ul luat în considerare fiind cel din finally.

16. Pe câte linii există erori în următoarea secvență de cod?

```
List<Integer> l1 = new LinkedList<Integer>();
List<Number> l2 = new LinkedList<Integer>();
List<Integer> l3 = new LinkedList<int>();
List<Number> l4 = new List<Integer>();
```

- 1
- 2
- **3**
- 4

R: Scopul exercițiului este verificarea regulilor legate de genericitate în contextul polimorfismului. Regula de bază, accentuată și în laborator este că atribuirile polimorfice se aplică doar tipului de bază nu și tipului generic. Din acest motiv, l2 și l4 sunt greșite. Atribuirea l3 este greșită deoarece tipurile primitive int/double/float/short/byte/boolean nu pot fi folosite ca tipuri generice.

17. Ce colecție ar fi cel mai bine de folosit dacă am vrea să menținem o serie de configurări/proprietăți ale aplicatiei, citite dintr-un fișier de configurare. Alegeți în funcție de cat de ușor e de lucrat cu colecția respectivă în cazul de față, al lizibiltății codului și eficiența d.p.d.v. al timpului de acces.

- ArrayList
- HashSet
- **HashMap**
- LinkedHashSet

R: Mai precizăm exemplul dat în timpul testului: fișierul de configurare poate conține date de genul: “os = linux” sau “graphics = low” etc. Scopul întrebării este verificarea înțelegerii avantajelor folosirii HashMap-ului și identificarea situațiilor în care e mai bine să îl folosiți (bad coding/design ar fi aici folosirea a doi vectori umpluți și parcurși în paralel, greșeală întâlnită în unele cazuri la laborator).

18. Care dintre afirmațiile următoare sunt adevărate in contextul limbajului Java?

1. Dacă a.equals(b) == false, atunci a.hashcode()==b.hashcode() este false.
2. Metoda equals trebuie implementată pentru a determina dacă două obiecte sunt egale.
3. Chiar dacă propriile obiecte nu suprascriu equals, ele pot fi folosite drept chei în obiecte de tip Map, fără a avea vreun caz de funcționare incorectă.
4. HashSet nu permite duplicate și nu menține ordinea elementelor.

- 1, 2, 3
- 1, 4
- 2, 3, 4
- **2, 4**

R: Scopul principal al acestui exercițiu este verificarea înțelegerii folosirii/implementării equals și hashcode. Afirmația 1 e falsă pentru că e fix negarea contractului Java pentru equals și hashcode. Afirmația a 3-a este falsă pentru că există situații în care dacă nu vă faceți propria implementare a metodei equals în clasa voastră folosită drept cheie în Map, atunci comportamentul programului poate sa nu fie cel dorit: se adauga obiecte drept chei in map care din punctul vostru de vedere sunt egale, insa neavand nici equals si nici hashcode cu implementari proprii, ele sunt considerate diferite. Aceasta afirmație verifică înțelegerea necesității de a suprascrie equals și hashcode pentru obiecte proprii, atunci când sunt adăugate în seturi sau folosite drept chei în map-uri.

19. Care pattern implică ascunderea vizibilității contructorilor unei clase?
- **Singleton**
- Visitor
- Observer
- Command

R: Ascunderea vizibilității constructorilor este modlitatea prin care ne asigurăm ca acea clasă nu poate fi instanțiată din altă parte a programului, deci ne ajută în a implementa conceptul de Singleton.

20. Pe care dintre următoarele pattern-uri l-am folosi, dacă în aplicație avem foarte multe notificări între componente?
- Visitor
- Command
- **Observer**
- Factory

R: Pattern-ul Observer este folosit pentru transmiterea de notificări de la componentele monitorizate către componentele care le monitorizează.

Grile POO 2015

21. Ce obținem la rularea următorului cod:

```
class A {
  public static boolean val;
  public A() { val = true; }
  public static void print() {
    System.out.println("First");
  }
}

public class Main {
  public static void main(String[] args) {
    A a = null;
    if (!a.val) a.print();
    else System.out.println("Second");
  }
}
```

- NullPointerException
- Second
- Eroare de compilare

- **First**

Explicație: Programul e corect sintactic, deci eroarea de compilare cade. Ideea principală este că se pot accesa membrii statici pentru obiectele a căror valoare e null. De ce funcționează: runtime-ul nu "dereferențiază" efectiv obiectul null, ci accesează membrii clasei obiectului, pe baza tipului declarat. De aceea, nu se aruncă excepție și se printează "First".

22. Care afirmații sunt corecte?

A. Statement-ul package poate fi pus doar pe prima linie necomentată dintr-un fișier
B. Specificatorii de acces pentru clase externe sunt doar public și default
C. Un fișier poate avea mai multe clase publice
D. Este obligatoriu pentru un fișier sa conțină statement-ul package
E. Un fișier poate avea mai multe statement-uri import

- A, B, C, E
- **A, B, E**
- B, C, E
- A, B, D, E

Explicație: să luăm afirmațiile pe rând: A) declararea pachetului poate fi doar prima linie necomentată de cod fără dubii, deci corect B) specificatorii pentru clase externe sunt doar public și default (adică fără modificator); ceilalți nu au sens decât în contextul unei alte clase (private, protected față de cine?) - corect C) un fișier poate avea o singură clasă publică - greșit D) dacă un fișier se află într-un pachet default, nu trebuie să aibă statement-ul package - greșit E) un fișier poate avea, desigur, mai multe import-uri (și este deseori cazul) - corect

23. Ce afișează următorul cod:

```
class Shape {
  public String type = " s ";
  public Shape() { System.out.print("shape ");}
}

public class Rectangle extends Shape {
  public Rectangle() { System.out.print("rectangle ");}
  public static void main(String[] args) {
    new Rectangle().go();
  }

  void go() {
    type = "r ";
    System.out.print(this.type + super.type);
  }
}
```

- rectangle r s
- **shape rectangle r r**
- rectangle r r
- shape rectangle r s

Explicație: Expresia new Rectangle() cheamă întâi super-constructorul, anume Shape(), care printează “shape”, după care se execută instrucțiunile din constructorul propriu al Rectangle, care printează “rectangle”. Apoi pe instanța de Rectangle se apelează metoda go din clasa Rectangle (ar trebui să fie clar), care setează type=“r”. Rectangle nu are un câmp type propriu, deci this.type și super.type vor avea aceeași valoare “r”. Deducem deci că se va afișa “shape rectangle r r”.

24. Ce afișează următorul program?

```
class Main {
  public static void main(String[] args) {
    Boolean b = new Boolean(true);
    if (b.equals(true)) {
      System.out.println("1");
      b = false;
      if (b == new Boolean(false)) {
        System.out.println("2");
      }
    }
  }
}
```

- **1**
- 12
- Eroare la compilare
- Nimic

Explicație: codul compilează; singura potențială problemă ar fi atribuirea unei valori boolean (tip primitiv) unei variabile Boolean (tip referință), dar știm că Java face autoboxing (adică se creează un nou Boolean cu false în interiorul lui). Esența întrebării este diferența dintre == și metoda equals. În mod clar b.equals(true) va fi adevărat, deci se va printa 1. Apoi, pentru că operatorul == compară referințele (adică verifică dacă cei doi operanzi sunt de fapt același obiect), b == new Boolean(false) va fi false, deci nu se va printa și 2.

25. Care variantă reprezintă suprascrierea corectă a metodei: protected int computeX(int a, float b) {...}?

- int computeX(int a, float b) {...}
- **public int computeX(int a, float b) {...}**
- protected int computeX(Integer a, Float b) {...}
- protected Integer computeX(int a, float b) {...}

Explicație: Întrebarea verifică cunoașterea regulilor pentru supracriere: * aceeași listă de argumente * același tip de return sau un subtip al acestuia * nu pot avea un modificator de acces mai restrictiv * pot arunca doar aceleași excepții, excepții derivate din acestea, sau excepții unchecked.

În cazul de faţă suprascrierea este corectă dacă se păstrează aceeași semnătură (tip de întors, nume + parametri, deci variantele 3 și 4 sunt greșite), iar modificatorul de acces este cel puțin protected. Modificatorul default poate sau nu să fie mai vizibil decât protected (exercițiu - gândiți-vă la scenarii), fapt pentru care compilatorul interzice prima variantă ca fiind suprascriere (i.e. dacă adăugați adnotarea @Override veți primi eroare).

26. Ce afișează următorul cod (Student e o subclasă a Person, iar Person conține metoda getName)?

```
// intr-o metoda main
Person s = new Student("Alice");
Person p = new Person("Bob");
InfoManager m = new InfoManager();
System.out.println(m.printInfo(s) +"; " + m.printInfo(p));

// in clasa InfoManager
public String printInfo(Person p){
  return "Person " + p.getName();
}

public String printInfo(Student s){
  return "Student " + s.getName();
}
```

- Person Bob; Student Alice
- **Person Alice; Person Bob**
- Student Alice; Student Bob
- Student Alice; Person Bob

Explicație: În lipsa altor detalii, getName() este aceeași metodă în clasele Person și Student, care întoarce câmpul de tip String din obiect, nici nu am mai scris-o în întrebare pentru că am fost siguri că înțelegeți scenariul. Ce e important de dedus este care metodă printInfo se apelează pentru cele două obiecte s și p. Răspunsul este simplu: se apelează acea metodă pentru care tipurile parametrilor coincid cu tipurile declarate ale obiectelor pasate efectiv. A nu se confunda cu runtime dispatch-ul metodelor suprascrise în clase derivate! În cazul nostru ambele obiecte erau declarate de tip Person, deci se apela de două ori metoda printInfo(Person), o dată pentru Alice, o dată pentru Bob.

Întrebarea testează că ştiţi că metodele supraîncărcate (overloaded) sunt 'alese' la compile-time, iar cele suprascrise (overriden) la runtime.

27. Ce se afișează?

```
class A {
  int x;
  public A() { init(0); }
  protected void init(int x) { this.x = x; }
}

class B extends A {
  public B() { init (super.x + 1); }
  public void init(int x) { this.x = x + 1; }
}

public class Test {
  public static void main(String[] args) {
    A a = new B();
    System.out.println(a.x);
  }
}
```

- 0
- 2
- **3**
- 1

Explicație: oh but why. De ce 3...

E clar că variabila a va avea tipul B la runtime. Ce e important e ce se întâmplă la construcția obiectului B. Constructorul lui B cheamă super-constructorul fără parametri, deși nu e scris explicit. Super-constructorul face init(0). Aici este foarte important. Se cheamă funcția init din B. De ce? Înainte să se apeleze efectiv constructorul B, runtime-ul alocă pe heap memorie aferentă unui obiect B și leagă toate numele metodelor la implementările efective, inclusiv init-ul suprascris. Toate acestea, înainte să se apeleze efectiv constructorul. Metoda init se va chema deci din clasa B, care face this.x = x+1. Asta înseamnă că în acest moment, a.x = 1. După apelul super-constructorului, se execută corpul propriu al constructorului B, care face init(super.x + 1). Pentru că super.x = this.x = 1, se cheamă deci init(2), care iarăși face this.x = 2 + 1, de unde this.x = 3. Cu această valoare se cedează controlul apelantului și deci a.x = 3.

Este recomandat ca în codul vostru să nu apelaţi metode polimorfice în constructori, tocmai din cauza unui astfel de comportament.

28. Care afirmații sunt corecte? (Ci denotă clase, Ii denotă interfețe)

A. C1 extends I1
B. I1 extends I2, I3

C. I1 implements I2
D. C1 implements I1,I2
E. C1 extends C2, C3.

- A, B, C, E
- C, E
- B, D, E
- **B, D**

Explicație: Este o întrebare standard despre fundamentele limbajului Java. Esențial este că o clasă poate extinde o singură altă clasă și poate implementa oricâte interfețe, iar o interfață poate extinde oricâte alte interfețe.

Afirmația E este deci greșită, deci pică din start 3 variante, ceea ce vă asigură răspunsul corect, dar să mergem mai departe. Afirmațiile A și C iarăși nu au sens, o clasă extinde altă clasă și o interfață extinde altă interfață. Afirmația D este corectă și apare des în viața reală. Ce este potențial tricky la această întrebare este afirmația B, care este corectă. De ce este corectă: în virtutea implementării mai multor interfețe, compilatorul este doar interesat de adunat semnături de metode în cadrul interfețelor. Vă puteți pune întrebarea ce se întâmplă dacă I1 și I2 conțin aceeași semnătură de metodă și pe care dintre ele o moștenește I. Un răspuns (cu niște simplificări) e că nu contează; ceea ce contează este ca acea semnătură de metodă să se afle în I. Deci faptul că o interfață extinde mai multe alte intefețe cu semnături de metode potențial identice nu deranjează compilatorul, pentru că nu există și implementări diferite care să declanșeze eventuale ambiguități.

29. Fie:

```
interface ITest {
  protected int x = 10;
  int y;
  int z = 20;
  abstract void foo();
  final int f(int x);
}
```

Care linii din corpul interfeței (numerotate de la 1 la 5) sunt corecte?

- 1,3,5
- **3,4**
- 1,2,3
- 4

Explicație: Esențial este faptul că orice variabilă din corpul unei interfețe este automat (dedus) public static final, iar metodele sunt implicit public abstract. Înarmați cu aceste cunoștințe, să luăm acum fiecare linie pe rând:

- protected și public (dedus) intră în contradicție - greșit
- y e final (implicit) și nu e inițializat - greșit
- Corect

- puteți considera declarația un pleonasm, dar compilatorul nu este deranjat, abstract este oricum dedus - corect
- metodele din interfețe sunt prin definiție abstract; final + abstract = compiler headbang - greșit.

30. Cu ce poate fi înlocuită linia (xxx) pentru a obține o instanță a B?

```
class A {
  class B {}
}

// in main
A a = new A();
(xxx)
```

- B b = new B();
- A.B b = new B();
- **A.B b = new A().new B();**
- A.B b = new a.B();

Explicație: Întrebare standard despre sintaxa Java la instanțierea unei clase interne nestatice. Corect este (obiect de tip A).new B(), deci singura variantă care respectă această sintaxă este cea din bold.

31. Fie interfața Runnable cu singura metodă public void run(). Clasa Thread are un constructor ce primește un Runnable ca parametru și expune o metodă public void start(). Ce concluzie trageți de la următorul cod?

```
new Thread(new Runnable() {
  public void run() {
    while(true) {
      System.out.println("Nyan cat!");
    }
  }
} ).start();
```

- eroare de compilare, public void run() nu poate fi implementat “pe loc”
- **compilează și rulează fără probleme**
- eroare de compilare, new Runnable() este incorect sintactic, Runnable este o interfață
- eroare de compilare, new Thread(...).start() este incorect sintactic

Explicație: scenariul din întrebare este un exemplu clasic de instanțiere a unei clase anonime (e un scenariu real, Thread și Runnable stau la baza multithreading-ului în Java). Expresia new Runnable() scrisă fără implementarea care ar urma ar fi într-adevăr o eroare de compilare (nu se pot instanția tipuri abstracte). Dar, ca și în explicația de la laboratorul de clase anonime, new Runnable() { public void run() {...} } spune de fapt compilatorului să creeze o nouă clasă care

implementează Runnable și oferă implementarea cu Nyan Cat (lol), după care și creează un obiect de tipul respectiv. Cât despre new Thread(…).start(), sintaxa e corectă.

32. Ce cuvânt cheie introdus la (xxx) va permite compilarea programului:

```
abstract class A {
  int x;
  abstract void set();
}

public class Test {
  public static void main(String args[]) {
    (xxx) int num = 30;
    A a = new A() {
      public void set() {
        x = num;
      }
    };
  }
}
```

- public
- **Nu este necesar niciun cuvânt cheie în plus pentru a permite compilarea corectă a programului.**
- final
- static

Explicație: Felul în care este folosită variabila o face effectively final si nu este nevoie de vreun cuvânt cheie în fața declarației ei. Dacă era modificată atunci era nevoie de cuvântul cheie final. Vedeți explicația din laboratorul de clase interne despre effectively final.

33. Dacă dorim să stocăm un șir de elemente fără duplicate într-o colecție fără să ne intereseze ordinea elementelor sau sortarea lor, clasa cea mai potrivită este

- Vector
- HashMap
- **HashSet**
- TreeMap

Explicație: Când auziți “fără duplicate”, primul impuls trebuie să fie o implementare de Set, deci deja Vector cade, iar HashSet este aproape sigur. Să ne uităm totuși și la celelalte variante. TreeMap ține minte chei comparabile, deci neinteresându-ne de ordinea elementelor, pică. HashMap poate fi de asemenea folosit ca și HashSet cu obiectele noastre ca și chei, dar pentru a-l folosi corect, trebuie să stocăm și valori asociate cheilor, cu toate că nu ne interesează efectiv. HashSet e deci varianta optimă.

34. Ce se întâmplă la rularea următorului cod:

```
String[] strings = {"hello", "world"};
Object[] objects = strings;           // linia 2
List<?> list = new ArrayList<Object>(); // linia 3
for (Object obj : objects) {
  System.out.println(obj);
  list.add(obj);                      // linia 6
}
```

- **eroare de compilare la linia 6**
- afișează “hello” și “world” pe linii diferite
- eroare de compilare la linia 2
- eroare de compilare la linia 3

Explicație: Linia 2 e corectă sintactic, deși dacă apoi încercăm ceva de tipul objects[1] = 1 vom obține un ArrayStoreException la runtime. Linia 3 e perfect în regulă, pentru că ArrayList implementează List și wildcard-ul se potrivește cu Object. Ce nu putem face e să apelăm metoda add pe variabila list, pentru că la compilare nu se cunoaște tipul obiectelor conținute în listă. Doar null poate fi folosit ca parametru, pentru că null aparține oricărui tip referință.

35. Care dintre următoarele variante este corectă (compilează)?

- ArrayList <Person>mylist = new ArrayList<Student>();
- List <Person>mylist = new ArrayList<Student>();
- **List <Student>mylist = new ArrayList<Student>();**
- ArrayList<Student>mylist = new ArrayList<Person>();

Explicație: Ideea cheie în această întrebare și în lucrul cu generics în general este că, de exemplu (exemplul nostru), dacă Student e subclasă a Person, atunci ArrayList<Student> nu e subclasă a ArrayList<Person>, deci atribuirea nu este corectă (un astfel de exemplu este și în laboratorul de genericitate).

Matching-ul tipului generic (cel dintre paranteze unghiulare) este făcut la compilare și singura variantă care trece de această verificare este varianta în bold. Am fi putut avea matching corect cu wildcard-uri, dar nu a fost cazul aici. Pentru mai multe detalii puteți să citiți despre covarianță și contravarianță și despre type erasure ca să înțelegeți motivele pentru care Java se comportă astfel.

36. Despre excepțiile unchecked:

- clasele lor nu pot fi extinse
- **pot să nu e declarate în clauzele throws ale metodelor și pot să nu fie prinse**
- trebuie declarate în clauza throws a metodelor care le pot arunca
- pot declarate în clauza throws a oricarei metode, dar nu pot fi prinse

Explicație: Excepțiile unchecked derivă din RuntimeException. Ca orice alt tip referință, se pot crea noi excepții unchecked după cum simțim nevoia, deci primul răspuns cade. Specificația lor include răspunsul de mai sus în bold, care este și cel corect. Excepțiile unchecked pot, dar nu

trebuie neapărat să fie declarate în clauzele throws și pot, dar nu trebuie neapărat să fie prinse de blocurile try-catch.

37.

```
class A {
  public int x = 0;
}

public A foo() {
  A a = new A();
  try { a.x = 1;
    throw new NullPointerException();
  } catch (Exception e) {
    a.x = 2;
    return a;
  } finally { a.x = 3; }
}
```

Ce se întâmplă la:

```
A a = foo();
System.out.println(a.x);
```

- 2
- excepție NullPointerException neprinsă, catch-ul prinde doar excepțiile checked
- 1
- **3**

Explicație: este o întrebare care necesită multă atenție și câteva cunoștințe cheie. La momentul aruncării excepției, a.x = 1. NullPointerException derivă din Exception, deci va fi prinsă. În acel moment, a.x devine 2 și se va marca obiectul a ca fiind cel pe care îl va întoarce funcția. Atenție. Doar marcat. Este pusă pe stivă referința într-un loc special, într-un mod asemănător în care apelați funcții la IOCLA: puneți într-un registru valoarea de întors (pe care o puteți modifica de oricâte ori) și abia când executați instrucțiunea ret se poate folosi în blocul apelant. Instrucțiunea din bytecode este areturn.

Nu se cedează controlul apelantului încă, pentru că există un bloc finally care va trebui executat (știm că finally se execută no matter what). Blocul finally setează a.x la valoarea 3 pe același obiect, adică pe cel care va fi apoi întors. Abia atunci se cedează controlul apelantului. Se va printa 3.

Nu aveați nevoie să cunoașteți toate detaliile, ci doar că nu se cedează controlul (nu se “iese din funcție”) imediat ce se execută o instrucțiune return ca în C, ci doar după ce s-a executat codul din finally, timp în care obiectelor care vor fi întoarse li se poate modifica starea internă. Cum e și cazul nostru.

Există mari șanse să vă loviți de problema aceasta în cod scris de alte persoane. Problema vă pregătește pentru acest scenariu și aceste cunoștințe vă pot salva ore sau zile pierdute.

38. Vrem să implementăm un framework de user interface. Cu ce design pattern am putea modela comportamentul de onClick → doSomething pentru un element de tip buton oarecare?

- Visitor
- Singleton
- Factory
- **Observer**

Explicație: Factory este un pattern de construcție de obiecte, nu unul comportamental cum a fost descris în întrebare. Visitor este un pattern în care se pot parcurge și procesa în mod polimorfic structuri de date diverse, iar Singleton permite crearea unei singure instanțe de un anumit tip, deci niciuna nu are legătură cu scenariul din întrebare. Observer, în schimb, se potrivește perfect, pentru că dacă țineți minte, la elementele noastre observabile (de tip MyArrayList) înregistram observatori care se declanșau (notificau) atunci când executam o operație pe ele. Așa dorim să implementăm și comportamentul butoanelor: se înregistrează listeneri, adică observatori, iar la click să se creeze un obiect (eveniment) cu care să fie notificați și fiecare să-și ruleze un mecanism intern pe acel eveniment (procesare de date, afișare de conținut, navigare etc). Este chiar modelul real pe care se bazează aproape toate elementele interactive de UI existente (web, mobile, jocuri etc).

39. Care cuvânt dintre următoarele este rezervat în Java?

- method
- unsigned
- **native**
- array

Explicație: Cuvintele “method” și “array” pică imediat. “Unsigned” nu există în Java, iar “native” denumește metodele pe care le puteți scrie pentru a interfața cu programele scrise special pentru sistemul vostru (arhitectura de calcul, sistem de operare etc). Consultați laboratorul de Reflection. Nu aveați nevoie de informații detaliate despre ce face Java Native Interface, ci doar că există astfel de metode.

40. Fie următorul test JUnit funcțional. Ce se va afișa în urma execuției lui?

```
public class Test {
 @Before
 public void before() { System.out.print("before:");}

 @Test
 public void test1() { System.out.print("test1:");}
```

```
    @After
    public void after() { System.out.print("after:");}

    @Test
    public void test2() { System.out.print("test2:");}
}
```

- before:test1:test2:after:
- test1:test2:
- before:test1:before:test2:
- **before:test1:after:before:test2:after:**

Explicație: întrebare simplă despre funcționarea JUnit. Fiecare test în parte este precedat de rularea metodei adnotată cu Before și succedată de rularea metodei adnotată cu After.

Grile POO 2016

41. Ce se va afișa la rularea următorului cod?

```
public static void main(String[] args) {
        String s = 0+1+"ONE"
                +3+2+"TWO"+"THREE"
                +5+4+"FOUR"+"FIVE"+5;
        System.out.println(s);
}
```

- 01ONE32TWOTHREE54FOURFIVE5
- **1ONE32TWOTHREE54FOURFIVE5**
- 1ONE5TWOTHREE9FOURFIVE5
- 01ONE5TWOTHREE9FOURFIVE5

R: Neavând paranteze sau alți operatori în afară de adunare, expresia se evaluează de la stânga la dreapta în ordinea dată. Întâi avem adunare de numere întregi (0+1), ceea ce dă tot un număr întreg (1). După aceea avem adunare între un număr întreg și un String, iar în Java nu trebuie conversie explicită pentru a face această operație, rezultatul fiind String-ul “1ONE”. După aceea toată expresia devine concatenare de String-uri, realizându-se conversia implicită a numerelor întregi la șiruri de caractere. Dacă adunările 3+2 sau 5+4 erau în paranteze, atunci se evalua întâi ceea ce era în paranteze, rezultând 5, respectiv 9, și se concatena la șirul de caractere.

Laboratorul 1 conține explicații și un exemplu (VeterinaryTest) referitoare la această conversia implicită. Detalii suplimentare pe acest subiect: Java Tutorial - Converting Between Numbers and Strings.

42. Ce metode sunt vizibile din clasa B?

```
package A;
```

```
class A {
   private void show1() {}
   protected void show2() {}
   void show3() {}
}

(alt fisier)
package B;
class B extends A {
   public void show2() {}
}
```

- show1, show2 din B, show3
- show2 din B, show2 din A, show3
- codul nu compilează datorită suprascrierii greșite
- **show2 din B, show2 din A**

R: Întrebarea verifică înțelegerea tipurilor de acces. Răspunsul corect este show2 din B și show2 din A pentru că show1 este privată iar show3 este default, deci nu poate fi vizibilă din alt pachet. Deoarece din bucata dată de cod lipsea import-ul clasei A, și putea induce în eroare (codul nu ar fi compilat și nici una din variante nu ar fi fost corecte) am decis să anulăm și această întrebare.

43. Fie următoarele clase:

```
A.java
public class A {
      int x;
      int y;

      A() {
            x = 1;
            y = 1;
      }

      A(int x) {
            this.x = x;
            this.y+=1;
      }
}

class B extends A {
      B(int x) {
            this.y += this.x + x;
```

```
        }
    }
```

Ce se afişează dacă rulăm:

```
    System.out.print(new A(1).y);
    System.out.print(new B(1).y);
```

- 20
- 23
- 24
- 35
- *** 13**

Deoarece întrebarea a fost formulată greșit (variantele de răspuns erau pentru altă versiune a codului), am decis să o anulăm, astfel toata lumea primind punctajul pentru acest subiect.

R: În cazul primului număr de afişat, se apelează constructorul cu parametru din clasa A, valoarea câmpului y devenind 1, deoarece, la crearea clasei, ea este 0 iar apoi este incrementată cu 1. Cea de-a doua valoare este egală cu 3 deoarece constructorul lui B apelează implicit constructorul fără parametrii din părintele său, A. Astfel, câmpul y din al doilea obiect este iniţializat cu 1 în constructorul fără parametrii din A, apoi incrementat cu valoarea câmpului x (iniţializat tot cu 1) şi cu parametrul x.

44. Ce valoare va returna b.show() ?

```
public class B {
   B b = new B();

   public int show () {
      return (true ? null : 0);
   }

   public static void main(String[] args)  {
      B b = new B();
      b.show();
   }
}
```

- java.lang.NullPointerException
- **java.lang.StackOverflowError**
- 0
- null

R: Execuția acestui cod va genera StackOverflowException datorită instanțierii recursive a clasei B (linia 1 din clasa B). Practic se va apela recursiv constructorul clasei B și nu se va ieși niciodată din vreun constructor, astfel umplându-se stiva de apeluri.

45. Care dintre următoarele clase sunt imutabile?
- public class Test { private final int x = 3 };
- Object, String
- **Integer, String**
- public final class Test { public int y; Test(int y) { this.y = y;} }

R: Obiectele imutabile (imutable) sunt obiecte care după creare nu mai pot fi modificate. Clasa String este imutable, conținutul ei nu poate fi modificat și nici nu poate avea subclase mutabile, fiind declarată final. Asta înseamnă că dacă dorim să modificăm un String (e.g. printr-o metodă de replace a unui caracter) obținem de fapt alt obiect String. La fel ca și String, clasa Integer, este imutabilă și nici nu poate fi extinsă, la fel ca și restul wrapperelor pentru primitive.

Legat de restul variantelor de răspuns:

- clasa Test pare imutabilă, deoarece câmpul x nu poate fi modificat, însă poate fi extinsă, ceea ce înseamnă că obiectele de tip Test pot să fie și mutabilă, totul depinzând de cum implementăm subclasele. Pentru mai multe detalii și un exemplu, urmăriți răspunsurile de pe acest thread.
- clasa Object nu este imutable, altfel toate obiectele ar fi imutable (toate clasele moștenesc Object)
- clasa Test este declarată final, adică nu poate fi extinsă, dar asta nu înseamnă că nu poate fi mutable. Având variabila publică y, această clasă poate fi schimbată, deci este mutable.

46. Care afirmație este corectă?
- **o clasă poate implementa mai multe interfețe, însă poate extinde o singură clasă abstractă**
- o clasă abstractă are cel puțin o metodă abstractă
- membrii unei clase abstracte sunt considerați \texttt{static final}
- în interfețe și clase abstracte nu pot fi definite metode statice

R: Această întrebare verifică noțiuni de bază legate de clase abstracte și interfețe. Prima variantă este corectă, e o restricție de bază a limbajului, o clasă putând extinde o singură clasă (fie ea abstractă sau nu) și poate implementa mai multe interfețe. Această restricție e bine de avut în vedere atunci când vă decideți dacă să faceți o clasă abstractă doar cu metode abstracte sau o interfață. Dacă clasa care o extinde are nevoie sa extindă și altă clasă atunci mai bine creați o interfață.

Legat de restul variantelor de răspuns:
- o clasă abstractă nu este obligată să conțină metode abstracte
- membrii unei clase abstracte poti fi declarați ca membrii oricărei clase, nu este obligatoriu să fie static final. La interfețe este obligatoriu să fie public static final.

- puteți defini metode statice atât în clasele abstracte (dintotdeauna) cât și în interfețe (din java 8). Chiar dacă nu erați siguri dacă pot fi declarate în interfețe, fiindcă aveam "și", afirmația era invalidată de partea cu clasele abstracte.

47. Care afirmaţii sunt corecte? (Ci denotă clase, Ii denotă interfeţe)

A. C1 extends I1
B. I1 extends I2, I3
C. I1 implements I2
D. C1 implements I1,I2
E. C1 extends C2, C3.

- A, B, C, E
- B, D, E
- **B, D**
- C, E

R: Acestă întrebare verifică noțiuni de bază legate de conceptul de moștenire și implementare interfețe. O clasă poate extinde o singură clasă. O clasă poate implementa oricâte interfețe. O interfață poate extinde oricâte alte interfețe. Doar B și D sunt afirmații corecte, celelalte fie folosesc keyword-ul greșit (extends în loc de implements și invers - A, respectiv C), fie prezintă moștenire multiplă (E).

48. Care variantă reprezintă supraîncărcarea corectă a metodei: String getMessage()

- public String getMessage()
- StringBuffer getMessage()
- **public String getMessage(String from)**
- public String getMessage() throws Exception

R: Această întrebare verifică cunoștiințe de bază ale conceptului de supraîncărcare (overloading). Regulile pentru o supraîncârcare corectă sunt prezentate și în laboratorul 6: metoda supraîncărcată are neapărat o listă diferită de argumente. Varianta a treia este singura care respectă această regulă. Schimbarea modificatorului de acces, a tipului de return sau a excepțiilor aruncate, fără a schimba numărul sau tipul parametrilor, nu reprezintă o supraîncărcare corectă, și nici nu este permisă la compilare.

49. Care dintre urmatoarele variante nu defineste încapsularea?

- expunerea unei interfețe high-level de lucru cu obiectul
- accesul la membri private folosind metode de tip getter și setter
- **posibilitatea suprascrierii (overriding) metodelor**
- construirea de obiecte complexe și ascunderea modului lor de funcționare

R: Variantele 1,2 și 4 reprezintă toate definiții/proprietăți ale încapsulării. Suprascrierea metodelor nu are nici o legătură cu conceptul de încapsulare, al cărui scop este ascunderea comportamentului intern al obiectului și oferirea unei interfețe de lucru cu acesta, controlând astfel accesul la variabilele sale interne, nepermițând modificarea lor din alte clase. De exemplu este bine să aveți variabilele private, sau cel mult protected și să oferiți getter și setteri pentru

accesarea lor. Acest lucru constituie un avantaj mai ales când aveți nevoie să faceți ceva suplimentar cu ele în getteri și setteri, de exemplu o validare.

50. Care combinație reprezintă, într-o clasă pe nume Test, o suprascriere, respectiv o supraîncărcare validă (overriding și overloading) pentru metoda equals din java.lang.Object?

- public Boolean equals (Object o) \ protected Integer equals (Object b)
- boolean equals(Object o) \ public boolean equals(Test t)
- public Boolean equals (Object t) \ public int equals (Object b)
- **public boolean equals(Object t) \ public int equals(Test t)**

R: Toate variantele, în afară de ultima nu respectă cel puțin un criteriu obligatoriu pentru o suprascriere sau supraîncărcare validă, prezentate și în laboratorul 6.

Metoda equals din clasa Object este publică, întoarce un boolean și primește ca parametru un Object. Fiindcă modificatorul de acces este public și altul mai puțin restrictiv nu există, în clasa Test nu puteți suprascrie decât dacă metoda este tot public. Tipul de return și semnătura trebuie păstrate identice. Tipul de return poate diferi doar când este înlocuit cu un subtip, ceea ce nu este cazul în acest exercițiu. O greșeală comună este de a pune în loc de Object, tipul clasei respective, in acest caz Test. Aceasta ar fi o supraîncarcare și nu o suprascriere. Tot o supraîncărcare este și dacă, pe lângă tipul parametrului schimbăm și tipul de return, ca în varianta de mai sus.

51. Ce se afișează la execuția codului următor (dacă se execută):

```
Package.java
public class Package {
   public String checksum() { return "Package"; }

   public static void main(String []args) {
      Comm c = new Comm();
      System.out.println(c.send((Package)new UDP()) + "; "
                  + c.send(new TCP()) + "; "
                  + c.send(new UDP()));
   }
}

class UDP extends Package {
   public String checksum() { return "UDP"; }
}

class TCP extends Package {
   public String checksum() { return "TCP"; }
}
```

```
class Comm {
   String send(Package p) { return "PKG:" + p.checksum(); }
   String send(UDP p) { return "UDP:" + p.checksum(); }
   String send(TCP p) { return "TCP:" + p.checksum(); }
}
```

- codul are o eroare de compilare
- UDP:UDP; TCP:TCP; UDP:UDP
- **PKG:UDP; TCP:TCP; UDP:UDP**
- PKG:UDP; PKG:TCP; PKG:UDP

R: La compilare, vor fi considerate 3 obiecte, unul de tip Package, unul de tip UDP și unul de tip TCP și se va considera apelul metodelor send pt Package, UDP și TCP. La runtime, obiectele p din metodele send vor fi de tip UDP, TCP și UDP. Practic, la runtime care metodă suprascrisă (overriden) să fie apelată, în timp ce la compilare se decid metodele supraîncărcate (overloaded).

52. Care din următoarele afirmații este adevarată despre clasele interne statice în Java:

- Este nevoie de o referință la un obiect din clasa externă pentru a putea instanția un obiect al clasei interne
- **Nu are acces la membrii non-statici ai clasei externe**
- Atributele și metodele conținute de aceasta trebuie să fie statice
- Trebuie neapărat să moștenească clasa externă

R: Scopul acestei întrebări este verificarea înțelegerii conceptelor de bază despre clasele interne statice și de a nu face confuzia între ele și cele nestatice. După cum ați aflat și în laboratorul despre clase interne, clasele interne statice pot fi instanțiate fără a avea nevoie de o referință a clasei externe. Acest lucru este similar cu accesul și alți membrii statici (variabile, metode) fară a avea nevoie de o instanță a clasei. Din cauză că nu avem o instanță a clasei externe, nu putem accesa membrii săi non-statici.

Legat de celelalte variante de răspuns:

- prima afirmație se referă la clasele interne ne-statice
- a treia afirmație prezintă o restricție care nu este impusă de aceste clase
- a patra afirmație, ca și a treia, este greșită și absurdă în contextul claselor interne

53. Cum afișăm y din Inner?

```
class Outer {
    int y;
    private class Inner {
       int y;
    }
}

class Outer1 extends Outer {
```

```
    public void show() { //todo afisare y}
}
```

- System.out.println(this.new Inner().y);
- System.out.println(super().new Inner().y);
- System.out.println(Outer.this.Inner().y);
- **nici una**

R: Să luăm pe rând variantele de răspuns și să vedem ce putem elimina:

System.out.println(this.new Inner().y); - incorect, nici nu compilează pentru că Inner este membru privat al clasei Outer, deci nu poate fi instanțiată decât din interiorul lui Outer.
System.out.println(super().new Inner().y); - incorect deoarece apelul super() poate fi făcut doar din constructorul lui Outer1. Dacă totuși nu ați fi observat că este call-ul pt constructor și l-ați fi confundat cu super.ceva_al_clasei_parinte, tot ar fi trebuit să ridice red flags accesarea clasei private Inner.
System.out.println(Outer.this.Inner().y); - dacă am fi fost într-o clasă internă lui Outer, atunci cu Outer.this am fi obținut referința clasei externe Outer. Doar că apelul acesta este în Outer1, care moștenește Outer, nu este internă ei, deci nu are sens, și nici nu compilează.
Rămâne astfel doar ultima variantă, “nici una”. Dacă vreți să îl afișați pe y este suficient să scrieți: System.out.println(y); sau System.out.println(this.y);

54. Care declarație este corectă?

- List<Integer> list = new List<Integer>();
- ArrayList<Integer> list = new List<Integer>();
- ArrayList<Object> list = new ArrayList<Integer>();
- **List<Integer> list = new ArrayList<Integer>();**

R: Scopul întrebării era să verificăm că știți să instanțiați corect o clasă parametrizată, în particular că generics nu sunt covariante și că nu puteți instanția o interfață. Primele două variante de răspuns sunt greșite pentru că se instanțează interfața List. A treia variantă este greșită datorită restricțiilor de genericitate. See also discuția despre genericitate și subtipuri din laboratorul despre genericitate și acest articol.

55. Ce colecție ar fi cel mai bine de folosit dacă am vrea să menținem o serie de configurări/proprietăți ale aplicatiei, citite dintr-un fișier de configurare. Alegeți în funcție de cat de ușor e de lucrat cu colecția respectivă în cazul de față, al lizibiltății codului și eficiența d.p.d.v. al timpului de acces.

- ArrayList
- HashSet
- **HashMap**
- LinkedHashSet

R: De exemplu fișierul de configurare poate conține date de genul: “os = linux” sau “graphics = low” etc. Scopul întrebării este verificarea înțelegerii avantajelor folosirii HashMap-ului și identificarea situațiilor în care e mai bine să îl folosiți (bad coding/design ar fi aici folosirea a doi vectori umpluți și parcurși în paralel, greșeală întâlnită în unele cazuri la laborator).

(subiectul acesta a fost dat și în testul din 2014)

56. Care afirmație este falsă?

- interfața Comparable<T> conține o metodă int compareTo(T o);
- interfața Comparator<T> conține o metodă int compare(T o1, T o2);
- **interfața Comparator<T> conține o metodă int compareTo(T o);**
- interfața Comparator<T> conține o metodă boolean equals(Object obj) ce face override metodei din Object;

R: The hint is in their names: interfața Comparable face ca obiectele de tip T care o implementează să devină comparabile între ele, în timp ce interfața Comparator permite clasei care o implementează să compare două obiecte de un anumit tip T. Subiectul acesta nu verifică că știți exact numele metodelor din aceste interfețe, ci că știți diferența dintre ele. Primele două afirmații sunt adevărate pentru că metoda de comparare are un singur parametru pentru Comparable și doi pentru Comparator. A treia varianta este cea falsă pentru că avem un singur parametru în metoda Comparator-ului.

57. Ce se va întâmpla la rularea următorului program?

```
Test.java
public class Test {
   public static int f(int i){
      try{
         System.out.print(i);
         return 1;
      } finally{
         try {
            return 10 / 0;
         } catch (Throwable e){
            return 100;
         } finally {
            System.out.print(7);
         }
      }
   }

   public static void main(String[] args){
      System.out.print(f(9));
   }
}
```

- 9100
- **97100**
- 91007

- 977100

R: Blocul finally se execută întotdeauna, chiar și dacă avem return (în afară de atunci când avem System.exit și se oprește masina virtuală). Din această cauză flow-ul codului de mai sus este: syso(9) → se intră în finally → return 10/0 generează eroare din cauza împărțirii la zero → eroare este de tip Throwable și este prinsă în catch → se excută finally-ul acestui bloc try-catch, syso(7) → se termină blocul al doilea finally → se termină primul bloc finally → se întoarce 100 → syso(100).

58. Care afirmație este corectă pentru codul următor?

```
ErrorTest.java
import java.io.*;
public class ErrorTest {
   void foo() throws IOException {
      throw new RuntimeException();
   }

   public static void main(String[] args) {
      ErrorTest err = new ErrorTest();
      try {
         err.foo();
      } catch (IOException e) {
         System.out.println("Caught exception");
      }
   }
}
```

- se va afişa Caught exception
- **eroare la rulare: RuntimeException**
- eroare la compilare: antetul metodei foo trebuie să includă şi RuntimeException în clauza throws
- eroare la compilare: trebuie adăugat un bloc catch şi pentru RuntimeException

R: În metoda foo se aruncă excepția RuntimeException și nu este prinsă în main, deci va apărea la rulare. Blocul catch prinde doar excepțiile de tip IOException. De asemenea, pentru că RuntimeException este excepție unchecked, programul compilează chiar dacă nu o includem în clauza throws a metodei foo sau dacă nu facem un catch pentru ea.

59. Ce design pattern ar fi util de folosit în cazul în care avem o colecție de date și dorim să facem o statistică relevantă în funcție de un anumit criteriu?

- **Visitor**
- Command
- Factory
- Strategy

R: Deci ce avem în această situație? O colecție de date, pe care presupunem că este reprezentată printr-o colecție (lista, de exemplu) de anumite obiecte, și dorim să facem operații pe ea pentru a obține diverse statistici. Asta sună extrem de similar cu un scenariu de folosire pentru Visitor. Factory pattern este pattern structural, ce se ocupă cu crearea obiectelor, deci nu prea are legatură cu situația dată (bine, putem folosi Factory pentru a crea acele obiecte din colecție, dar nu este atât de relevant pentru întrebarea noastră). Command nu prea se potrivește direct, decât dacă încercăm să abordăm problema altfel, să decuplăm aplicarea unor comenzi de prelucrare, iar Strategy ar putea fi o opțiune potrivită dacă am vrea să alegem dinamic, la runtime, dintre mai multe feluri de a face acea statistică.

60. Ce design pattern folosim dacă vrem să simulăm pasarea și procesarea de “pointeri la funcții” încapsulați în obiecte?
- **Command**
- Factory
- Singleton
- Observer

R: Factory și Singleton pică din start pentru că sunt pattern-uri legate de crearea obiectelor, iar întrebarea nu dorește soluționarea acestui aspect. Componentele și relațiile sugerate de pattern-ul Observer nu se potrivesc cu această problemă, nu dorim să monitorizăm obiecte, nu dorim ceva event-triggered etc. Pattern-ul Command este cel mai potrivit pentru această situație, fiecare pointer fiind un obiect de tip comandă.

Grile POO 2017

61. Care dintre urmatoarele metode reprezinta variante corecte de supraıncarcare a void act()?
    1. void act(int i)
    2. int act()
    3. void act(int i) throws Exception
    4. void act() throws Exception {throw new Exception();}
- 1, 2, 3, 4
- 3, 4
- **1, 3**
- 1, 2

62.

```
package pkg1;
public class Foo {
        public int a = 5;
        protected int b = 6;
        int c = 7;
}
---alt fisier:
```

```
package pkg2;
import pkg1.*;
public class Bar {
        public static void main(String[] args) {
                Foo f = new Foo();
                System.out.println(f.a);
                System.out.println(" " + f.b);
                System.out.println(" " + f.c);
        }
}
```

- 5 6 7
- **programul are 2 erori de compilare**
- programul are o eroare de compilare
- 5 urmat de o exceptie

63. Care este diferenta ıntre clase interne statice(CIS) si nestatice(CIN)?
- CIN sunt vizibile doar ın clasa exterioara acestora. CIS sunt vizibile oriunde ın program
- CIN pot fi instantiate doar ın clasa exterioara. CIS pot fi instantiate oriunde ın program
- CIS nu pot fi mostenite, ın schimb cele nestatice da
- **CIN nu pot fi instantiate fara crearea, ın prealabil, a unei instante a clasei exterioare**

64.

```
class B {
        public static void foo() {
                System.out.println("1");
                this.foo();
        }
        private void foo(int x) {
                System.out.println("2");
        }
}
public class A extends B {
        public static void foo() {
                System.out.println("3");
        }
        public static void main(String[] args) {
                B.foo();
                A.foo();
        }
}
```

- Eroare la linia static void foo() din clasa A
- Se afiseaza 1 2 3 pe linii separate
- Eroare la linia public static void foo() din clasa B

- **Eroare la linia this.foo()**

65. Care dintre urmatoarele patternuri permite crearea de obiecte fara a expune logica lor clientilor?
- Singleton
- Observer
- **Factory**
- Visitor

66. Care dintre urmatoarele afirmatii sunt adevarate?

1. Un fisier sursa poate contine oricate definitii de clase, cel mult una putand fi marcata public.

2. Un fisier sursa poate contine oricate clase marcate private.

3. Un fisier sursa poate contine oricate interfete, atat timp cat nu exista si definitii de clase ın respectivul fisier.

4. In cadrul unui fisier sursa instructiunile “import” nu pot succeda definitia unei clase sau interfete.

- 1,3
- 1,2
- 1,2,3,4
- **1,4**

67. Care dintre urmatoarele interfete nu extinde interfata Collection?
- Set
- **Map**
- List
- Queue

68. Cu ce poate fi ınlocuita linia 7 pentru a obtine o instanta a clasei B?

```
class A {
        class B {}
}
class Test {
        public static void main(String args[]) {
                A a = new A();
                /* Linia 7 */
        }
}
```

- A.B b = new A.new B();
- **A.B b = new A().new B();**
- A.B b = new A.B();
- B b = new B();

69. Cate linii sunt incorecte?

```
interface MyStruct<E> {
        static String name = "MyStruct";
        long ID;
        public void setElement(E e);
        abstract E getElement();
        protected void print();
}
```

- **2**
- 0
- 3
- 1

70. Care dintre afirmatiile de mai jos caracterizeaza supraıncarcarea si suprascrierea?

A. Suprascrierea se refera la definirea, ıntr-o clasa copil, de metode cu acelasi nume, numar si tip de parametri, ca ın clasa parinte;
B. Supraıncarcarea se refera la definirea de metode cu acelasi nume, ınsa tip de retur si parametri diferiti;
C. Suprascrierea se refera la ascunderea implementarii unor metode dintr-o clasa parinte, prin modificarea parametrilor acesteia;
D. Supraıncarcarea se refera la definirea mai multor metode cu acelasi nume, ınsa numar si/sau tip de parametri diferiti;

- **A si D**
- C si D
- C si B
- A si B

71.

```
class A {
        public int a = 1;
        float b = 2.0f;
        protected double c = 3.0d;
        private char d = 'x';
}
class B extends A { B () { c = a + b + d; }}
```

- programul nu va genera eroare la compilare
- eroare la compilare din cauza ca modificatorii de acces ai variabilelor a, b, d sunt diferiti
- **eroare la compilare din cauza ca variabila d nu poate fi accesata ın clasa B**
- eroare la compilare din cauza ca nu putem aduna o variabila de tip char, cu una de tip float

72. Ce pattern se foloseste daca se vrea adaugarea facila de noi functionalitati in viitor, fara modificarea structurilor existente?

- Factory
- Singleton
- **Visitor**
- Pizza

73.

```
try {
        try {
                System.out.print("A");
                throw new Exception("1");
        } catch (Exception e) {
                System.out.print("B");
                throw new Exception("2");
        } finally {
                System.out.print("C");
                throw new Exception("3");
        }
} catch (Exception e) {
        System.out.print(e.getMessage());
}
```

- **ABC3**
- ABC
- AB2
- AB2C3

74.

```
public class Parent {
        public Parent(String s) { System.out.println("B"); }
        public static void main(String[] args) {
                System.out.println("C");
                Child c = new Child("C");
        }
}
class Child extends Parent {
        public Child(String s) { System.out.println("D"); }
}
```

- programul va afisa ”DB”
- programul va afisa ”BD”
- programul va afisa ”C” urmat de o exceptie
- **eroare la compilare**

75. Care este un apel corect al metodei foo?

```
class C { public void foo() { } }
class B { public static class D extends C { } }
```

- new B.D.foo()
- **new B.D().foo()**
- B.D.foo()
- new B.D().super.foo()

76. Care varianta este corecta pentru a completa TODO-ul?

```
class Foo{
        //TODO
        public void f(int n, int t){
                ArrayList<Float> y = new ArrayList<Float>();
                for (int i = 0; i < 10; i++)
                        y.add(i * t * 1.0f);
                x.put(n, y);
        }
}
```

- **Map<Integer, ArrayList<Float>> x = new HashMap<Integer, ArrayList<Float>>();**
- HashMap<Integer, ArrayList<Float>> x = new HashMap<Number, ArrayList<Number>>();
- Map<Integer, List<Float>> x = new Map<Integer, ArrayList<Float>>();
- HashMap<Integer, List<Float>> x = new HashMap<Integer, ArrayList<Float>>();

77. Identificati afirmatiile corecte:

1. Un constructor nu poate avea un specificator de acces atasat definitiei.
2. O clasa poate implementa mai multe interfete, ınsa poate extinde o singura clasa abstracta.
3. Interfetele pot implementa alte interfete.
4. Dupa instantierea unui vector new Object[10] toate elementele au valoarea null.

- **2,4**
- 1, 2
- 1, 2, 3
- 1,2,4

78.

```
class Animal {
        Animal() { System.out.println("x "); }
}
class Horse extends Animal {
        Horse() { System.out.println("v "); }
        Horse(String race) {
                this();
                System.out.println("xf " + race);
```

```
        }
        public static void main(String[] args) {
            Horse h = new Horse("b ");
        }
    }
```

- xf b
- **x v xf b**
- v xf b
- xf b v x

79. Care dintre urmatoarele implementari ale interfetei Map mentine cheile sortate?

- HashMap
- LinkedHashMap
- **TreeMap**
- Hashtable

80.

```
    public class Test {
        public static int hash(Object o){
            return o.hashCode();
        }
        public static void main(String args[]){
            try {
                System.out.print("A");
                int x = hash(null);
                System.out.print("B");
            }
            catch(RuntimeException e){
                System.out.print("C");
            }
            catch(Exception e){
                System.out.print("D");
            }
            finally{
                System.out.print("E");
            }
    }
}
```

- niciuna dintre variante
- **ACE**
- ADE
- ABE

Grile POO 2018

81. Care dintre următoarele cuvinte cheie realizează moștenirea în Java?

- implements
- inherits
- **extends**
- super

R: Cuvântul cheie extends este folosit pentru relația de moștenire dintre clase sau dintre interfețe. implements este folosit când o clasă implementează o interfață, iar super pentru a apela constructorii clasei extinse sau metodele acesteia.

82. Care dintre următoarele concepte reprezintă o relație HAS-A?

- moștenirea
- polimorfismul
- **agregarea**
- încapsularea

R: Agregarea reprezintă o relație HAS-A (vedeți detalii în laboratorul Agregare și moștenire), o clasă ținând referință către obiecte ale altei clase. Moștenirea este o relație IS-A, polimorfismul reprezintă abilitatea unei clase să se comporte ca o altă clasă de pe lanțul de moștenire, iar încapsularea se referă la felul în care împachetăm informațiile dintr-o clasă (ce se poate accesa din exterior).

83. Care variantă reprezintă o supraîncărcare corectă pentru metoda: protected int getGrade(String course)

- protected int getGrade(String course) throws IOException
- private int getGrade(String course)
- protected long getGrade(String course)
- **public long getGrade(int studID)**

R: Regula de bază a suprîncărcarii(overloading-ului) este că numele metodei rămâne același dar antetul acesteia se schimbă, adică diferă numărul parametrilor și/sau tipul acestora. Variantele de răspuns greșite au lăsat parametrul identic, schimbând doar modificatorul de acces, excepțiile aruncate și tipul de return, acest lucru nefiind suficient. Detalii despre overloading găsiți în laboratorul More OOP & Visitor Pattern.

84. Ce se afișează?

```
public class Test {
   public static void main(String []args) {
      Drink tea = new Tea();
      tea.make();
   }
}
```

```
class Drink {
   public static void make() {
      System.out.println("Making drink");
   }
}

class Tea extends Drink {
   public static void make() {
      System.out.println("Making tea");
   }
}
```

- Making tea
- **Making drink**
- eroare la compilare, metoda make nu poate fi suprascrisă
- excepție la rulare

R: Metodele statice țin de clasă și nu de instanță, deci pentru ele nu are sens suprascrierea (overriding-ul). La execuția main-ului se va apelea metoda make din clasa Tea, același comportament obținându-l și dacă apelam Drink.make(). Sintaxa Java permite să apelați metodele statice și pe instanță dar nu recomandăm acest lucru pentru că afectează lizibilitatea codului, putând fi confundate la o citire rapidă cu metodele ne-statice.

85. Ce se afișează?

```
public class BasicInit {
      private int x;
      private boolean flag;
      protected String s;
      @Override
      public String toString() {
            return x + " " + flag + " " + s;
      }
      public static void main(String []args) {
            BasicInit basicInit = new BasicInit();
            System.out.println(basicInit);
      }
}
```

- 0 true
- null true
- **0 false null**
- null false null

R: Java inițializează variabilele instanțelor cu valori default: tipurile primitive primesc 0, boolean iar referințele la obiect cu null (vedeți laboratorul Constructori și referințe).

86. Ce se afișează?

```
class Device {
   public Device() {
      System.out.print("D");
   }
}

public class Watch extends Device {
   public Watch() {
      System.out.print("W");
   }

   public Watch(String name) {
      this();
      System.out.print(name);
   }

   public static void main(String []args) {
      new Watch("F");
   }
}
```

- WF
- **DWF**
- DW
- DF

R: Se afișează DWF datorită apelului constructorilor pe ierarhia de moștenire. În constructorul cu parametru din Watch se apelează explicit constructorul său fără parametru (apelul this()), iar în acesta se apelează implicit constructorul clasei părinte, Device, care apelează implicit constructorul din Object. Apoi ne întoarcem pe lanțul de apeluri și se afișează D, apoi W, apoi F.

87. Ce înseamnă constructorul implicit (default)?

- constructor fără parametri declarat de utilizator
- **constructor fără parametri adăugat de Java dacă nici un constructor nu a fost declarat**
- constructor fără implementare
- constructor fără modificatori de acces

R: Dacă nu adăugați vreun constructor în clasa pe care ați creat-o, atunci Java vă adaugă un constructor default, vedeți laboratorul Constructori și referințe.

88. Ce cuvinte cheie pot fi folosite pentru membrul MY_VALUE?

```
public interface Status {
     /* insert qualifier here */ int MY_VALUE = 10;
}
```

- final, static, private
- final, static, protected
- final, private, abstract
- **final, static, public**

R: Implicit, variabilele declarate în interfețe sunt final, static și public. Atunci când le declarați puteți scrie toate aceste cuvinte cheie sau le puteți omite (IDE-ul chiar vă sugerează să le omiteți). Ele sunt publice pentru ca este vorba de o interfață. Sunt statice din motive de încapsulare, altfel am declara variabile ale instanței publice. Sunt finale pentru a ne asigura că vor fi constante și nu li se vor atribui alte valori (e.g. dacă ar fi doar public static, atunci două sau mai multe clase care implementează interfața vor putea modifica la comun acea variabilă).

89. Care variantă definește cel mai bine legătura dintre interfețe și clase?

- Atât clasele, cat și interfețele definesc modul în care un obiect execută o operație
- **Interfețele precizează operațiile expuse de un obiect, în timp ce clasele modul în care acesta le execută**
- Nici clasele, nici interfețele nu precizează modul în care un obiect execută o operație
- Nu pot exista relații între clase și interfețe

R: Self-explanatory

90. Dacă B extinde clasa abstractă A și C extinde B, atunci care instanțiere este corectă?

- C cb = new B();
- B ba = new A();
- **A ab = new B();**
- C ca = new A();

R: Varianta corectă este A ab = new B(). Restul variantelor sunt incorecte pentru că nu putem atribui unei referinte de un anumit tip un obiect al supertipului. Dacă s-ar permite așa ceva, atunci la runtime, când apelăm o metodă a lui C, obiectul nu o va avea, pentru ca e de tip B. Încă ceva greșit în acele variante este că încercăm să instanțiem o clasă abstractă, iar acestea, ca și interfețele, nu se instanțiază.

91. Care din următoarele afirmații sunt adevarate despre clase interne statice?

A. Nu au acces la membri non-statici ai clasei exterioare
B. Este nevoie de o instanță a clasei externe pentru a o instanția
C. Trebuie sa moștenească clasa exterioară
D. Trebuie instanțiată astfel: Outer out = new Outer(); Inner in = out.new Inner();

- **A**
- B, D
- A, B
- C

R: Clasele interne statice (nested classes) nu au nevoie de o instanță a clasei externe, la fel ca și câmpurile și metodele statice. Din acest motiv nici nu pot avea acces la membrii non-statici ai acesteia, afirmația A fiind corectă și B incorectă. Afirmația C este falsă, nu există această restricție. Afirmația D este falsă datorită exprimării “trebuie”.

92. Care dintre următoarele afirmații este adevărată cu privire la clasele anonime?
- clasele anonime trebuie să aibă în constructor o referință către clasa exterioară
- clasele anonime trebuie să definească explicit constructorul fără parametri
- **claselor anonime nu li se poate declara un constructor**
- niciuna din variante

R: Clasele anonime nu au constructor, în laboratorul Clase interne puteți vedea mai multe detalii și exemple.

93. Ce se afișează?

```
Set<Integer> mySet = new LinkedHashSet<>();
mySet.add(1);
mySet.add(10);
mySet.add(100);
System.out.println(mySet);
```

- [10, 1, 100]
- [100, 10, 1]
- **[1, 10, 100]**
- numerele vor fi afișate într-o ordine arbitrară

R: Implementările interfeței Set variază în funcție de următoarele caracteristici: dacă oferă un set ordonat sau nu, sau daca oferă un set sortat sau nu. Varianta HashSet oferă un set neordonat și nesortat. LinkedHashSet însă reține ordinea în care s-au adăugat elementele, fiind implementată folosind o listă dublu înlănțuită.

94. Ce colecție ar fi mai eficientă de folosit dacă dorim să stocăm o secvență de elemente pe care să o modificăm rar dar pe care să o accesăm foarte des?
- LinkedList
- **ArrayList**
- Vector
- niciuna din variante

R: ArrayList este mai eficientă de folosit dacă accesăm des elementele din ea pentru că are complexitate O(1) pt acces. Implementările intefeței List variază în funcție de structura de date folosită pentru reținerea elementelor, astfel oferind complexități diferite pentru acces și modificări, fapt menționat și în laboratorul Colecții. În plus, una din implementări, Vector, este ineficientă deoarece toate metodele sale sunt sincronizate, permițând unui singur thread să le acceseze la un moment dat (este și depracated pentru că acum există alternative thread-safe mai eficiente e.g. Collections#synchronizedList).

95. Care instanțiere este corectă?
- Set<Integer> set = new HashSet<Object>();
- HashSet<Integer> set = new Set<Integer>();
- **Set<Integer> set = new HashSet<Integer>();**

- HashSet<Object> set = new HashSet<Integer>();

R: Varianta corectă este Set<Integer> set = new HashSet<Integer>();. Prima și ultima variantă sunt incorecte pentru că nu avem covarianță în cazul genericității, după cum este menționat și în laborator, dacă ChildType este un subtip (clasă descendentă sau subinterfață) al lui ParentType, atunci o structură generică GenericStructure<ChildType> nu este un subtip al lui GenericStructure<ParentType>. Această restricție se datorează faptului că genericitatea este aplicată la compilare, iar la runtime nu se fac verificări de tipuri. A doua variantă este greșită pentru că Set e super tipul lui HashSet și Set este și interfață, deci nu poate fi instanțiată.

96. Ce este JUnit?

- un framework pentru integration testing
- **un framework pentru unit testing**
- un framwork pentru fuzzy testing
- un framework pentru system testing

R: JUnit este framework-ul pentru unit testing studiat la laborator. Chiar dacă JUnit poate fi folosit împreună cu alte tool-uri și pentru integration tests (testează mai multe componente, e dependent de sisteme externe, e.g. o bază de date), scopul lui principal este unit testing-ul, iar la laborator s-a folosit pentru acest lucru.

97. Ce se afișează?

```
public class Test {
   int count = 0;
   void modifyCount() throws Exception {
      try {
         count++;
         try {
            count++;
            try {
               count++;
               throw new Exception();
            }
            catch(Exception e) {
               count++;
               throw new Exception();
            }
            finally {
               count++;
            }
         }
         catch(Exception e) {
            count++;
         }
```

```
        }
        catch(Exception e) {
            count++;
        }

    }
    public static void main(String[] args) throws Exception {
        Test test = new Test();
        test.modifyCount();
        System.out.println(test.count);
    }
}
```

- **6**
- 4
- 5
- 7

R: Întrebarea aceasta verifică înțelegerea conceptului de finally dintr-un bloc de prindere a excepțiilor. Ca și în alte limbaje (e.g. C#, Python, Javascript), și în Java, finally se execută de fiecare dată, după execuția blocului try sau catch. Urmărind codul, avem: try - count=1 —> try - count=2 —> try - count=3 —> catch count=4 —> finally - count=5 —> catch (pt ca primul catch aruncă excepția) count=6.

98. Care dintre următoarele patternuri ar fi mai util dacă dorim ca anumite clase să fie notificate de schimbări ale altor clase?

- Visitor
- Strategy
- Factory
- **Observer**

R: Folosind Observer putem avea obiecte “observabile” care atunci când își modifică starea să notifice alte obiecte dependente de acestea “observatori”.

99. Ce se afișează?

```
public class NetworkService {
    private static NetworkService instance
                    = new NetworkService();
    public String str;

    private NetworkService() {
        str = ""Hello, I am a string"";
    }

    public static NetworkService getService() {
```

```
        return instance;
    }

    public static void main(String[] args) {
        NetworkService a = NetworkService.getService();
        NetworkService b = NetworkService.getService();

        System.out.println(a.str == b.str);
    }
}
```

- **true**
- false
- 0
- 1

R: NetworkService este implementat ca un Singleton, existând o singură instanță a sa. Variabilele a și b reprezintă același obiect, deci comparația referințelor == este true.

100. Pentru a modela mai multe butoane din interfața grafică cu o anumită funcționalitate (ce este executată la apăsare) și diverse proprietăți (icon, text, animație), este o buna idee de a folosi urmatorul design pattern:

- Visitor Pattern
- Singleton Pattern
- Factory Pattern
- **Command Pattern**

R: Folosind Command Pattern putem modela ușor operațiile efectuate sub forma unor comenzi. Butoanele din GUI (interfața grafică a utilizatorului) și elementele din meniuri sunt de multe ori obiecte de tip Command. În plus față de abilitatea de a efectua comanda dorită, o acțiune poate avea o pictogramă asociată, un text sau altele. Hint ar fi trebuit să fie cuvântul “executată” , care duce cu gândul la metoda execute() specifică pattern-ului.

Grile POO 2019 CA

101. Care dintre urmatoarele afirmații sunt adevarate despre pachete în Java?

1) Orice clasă aparține unui pachet

2) Toate clasele dintr-un fișier aparțin aceluiași pachet

3) Daca nu se specifică niciun pachet, clasele vor aparține unui pachet special fără nume

4) Daca nu se specifică niciun pachet, se creează un nou pachet cu numele folderului în care se află clasa, iar aceasta va aparține acestui pachet

- 1, 2, 4
- **1, 2, 3**

- 1, 3
- 2, 4

102. Care este rezultatul programului?

```
public class Fruit {
   Fruit() {
      System.out.print(" Fruit ");
   }

   public static void main(String[] args) {
      Fruit cherry = new Cherry(" Sour ");
   }
}

class Cherry extends Fruit {
   Cherry() {
      System.out.print(" Cherry ");
   }

   Cherry(String type) {
      this();
      System.out.print(type + " Cherry ");
   }
}
```

- **Fruit Cherry Sour Cherry**
- Fruit Sour Cherry
- Cherry Sour Cherry
- Sour Cherry Cherry Fruit

103. Câte interfețe poate extinde o interfață în Java?

- una singura
- **oricâte**
- o interfață implementează alte interfețe, nu le extinde
- niciuna

104. De câte ori se realizează method overriding (suprascriere) și de câte ori method overloading (supraîncarcare)?

```
class Dog {
   void bark() {
      System.out.println("This dog is barking...");
   }
}
```

```
class Husky extends Dog {
   void bark() {
      System.out.println("Husky is barking...");
   }

   void bark(int time) {
      System.out.println("Husky is barking for "+time+" minutes");
   }

   int bark(String food) {
      System.out.println("Husky wants " + food);
      return 0;
   }
}
```

- **1 overriding si 2 overloading**
- 2 overriding si 1 overloading
- 1 overriding si 1 overloading
- 3 overriding si 1 overloading

105. Ce afișează următorul program?

```
public class Coffee {
   public static void main(String[] args) {

      CoffeeMaker coffeemaker = new StarbucksCoffeeMaker();
      Coffee espresso = new Espresso();
      Cappuccino cappuccino = new Cappuccino();

      coffeemaker.makeCoffee(espresso);
      coffeemaker.makeCoffee(cappuccino);
   }
}

class Espresso extends Coffee {
}

class Cappuccino extends Coffee {
}

class CoffeeMaker {
   public void makeCoffee(Coffee coffee) {
      System.out.println("Making coffee");
   }
```

```
    public void makeCoffee(Espresso coffee) {
        System.out.println("Making espresso");
    }

    public void makeCoffee(Cappuccino coffee) {
        System.out.println("Making cappuccino");
    }
}

class StarbucksCoffeeMaker extends CoffeeMaker {
    public void makeCoffee(Espresso coffee) {
        System.out.println("Making sugar espresso");
    }

    public void makeCoffee(Cappuccino coffee) {
        System.out.println("Making sugar cappuccino");
    }
}
```

- **Making coffee Making sugar cappuccino**
- Making espresso
  Making cappuccino
- Making sugar espresso
  Making sugar cappuccino
- Making coffee
  Making cappuccino

106. Care este valoarea de adevar a următoarelor afirmații? Clasele interne îmbunătățesc încapsularea. Clasele interne anonime trebuie să definească explicit constructorul fără parametri.

- Adevarat, Adevarat
- **Adevarat, Fals**
- Fals, Adevarat
- Fals, Fals

107. Care dintre următoarele colecții nu sunt iterabile (nu implementează patternul Iterable oferit în Java prin interfața Iterable)?

- ArrayList
- **HashMap**
- Queue
- Set

108. Ce afișează următorul program?

```
class Test {
   public static void main(String[] args) {
      System.out.println(breakingStuff());
   }

   public static int breakingStuff() {
      try {
         try {
            throw new Exception();
         } catch (Exception e) {
            return 1;
         } finally {
            return 2;
         }

      } catch (Exception e) {
         return 3;
      } finally {
         return 4;
      }
   }
}
```

- 1
- 2
- 3
- **4**

109. Visitor Pattern si Observer Pattern, din punct de vedere al clasificării Gang of Four, sunt:

- Visitor este un design pattern behavioural, iar Observer este structural
- Visitor este un design pattern structural, iar Observer este behavioural
- **ambele sunt behavioural**
- ambele sunt structural

110. Stabiliți valoarea de adevăr a următoarelor propoziții: Metodele statice nu pot fi suprascrise. Clasele interne statice nu pot fi extinse.

- Fals, Fals
- Fals, Adevarat
- **Adevarat, Fals**
- Adevarat, Adevarat

Grile POO 2019 CD

111. Ce va afișa următorul program?

```
class Test {
   public static void main(String args[]) {
      String s1 = "Wow, am luat 10 la grila la POO!";
      String s2 = new String(s1);
      System.out.println((s1 == s2) + " " + s1.equals(s2));
   }
}
```

- true true
- false false
- true false
- **false true**

112. Care dintre urmatoarele afirmații este corectă?

- Cuvântul final în fața unei metode arată faptul că acea metoda poate fi apelată doar în clasa respectivă
- O clasa final nu poate fi instanțiată
- **Cuvantul final în fața unei variabile arată că acea variabilă poate fi assignata o singura dată**
- Toate cele de mai sus

113. Care dintre urmatoarele afirmații despre clase abstracte în Java sunt FALSE:

1) Daca derivăm o clasa abstractă și nu implementam toate metodele abstracte, atunci clasa derivată trebuie de asemenea sa fie abstractă
2) Clasele abstracte pot avea constructori
3) O clasă nu poate fi abstractă fără o metoda abstractă
4) O clasă abstractă poate moșteni de la mai multe clase abstracte

- 1, 2
- 1, 3
- **3, 4**
- 1, 3, 4

114. Se dau următoarele clase:

```
class Lion {}
class LionKing extends Lion {}
class SimulateAnimal {
      protected Lion roar(char a) { //do stuff }
}
class Game extends SimulateAnimal {
```

```
        // TODO suprascriere roar
    }
```

Care dintre următoarele reprezintă o suprascriere corectă a metodei roar?

- protected LionKing roar(int a)
- private Lion roar(char a)
- **public LionKing roar(char a)**
- public LionKing roar(char a, int b)

115. Ce afișează următorul program?

```
class Dacia {
  String model = "duster";

  public Dacia() {
    printModel();
  }

  void printModel() {
    System.out.print(model);
  }
}

public class Ferrari extends Dacia {
  String series = "f40";

  void printModel() {
    System.out.print(series);
  }

  public static void main(String[] args) {
    Ferrari myCar = new Ferrari();
  }
}
```

- duster
- f40
- **null**
- duster f40

116. Care este valoarea de adevar a următoarelor afirmații? O clasa internă anonimă poate extinde o singură clasă sau poate implementa o singură interfață. O clasă internă metodelor nu poate fi abstractă.

- Adevarat, Adevarat
- **Adevarat, Fals**

- Fals, Adevarat
- Fals, Fals

117. Care afirmație despre LinkedHashSet din API-ul Java pentru colecții este adevarată?
- nu există clasa LinkedHashSet
- **pastrează ordinea de inserare a elementelor și nu permite duplicate**
- pastrează perechi de forma (Key, Value) și permite duplicate
- este o listă simplu înlănțuită unde fiecare element este o pereche (Key, Value)

118. Fie:

```
class A {
    public int x = 0;
}

public A foo() {
    A a = new A();
    try {
        a.x = 1;
        throw new NullPointerException();
    } catch (Exception e) {
        a.x = 2;
        return a;
    } finally {
        a.x = 3;
    }
}
```

Ce se întâmplă la:

```
A a = foo();
System.out.println(a.x);
```

- 1
- 2
- **3**
- excepție NullPointerException neprinsă, catch-ul prinde doar excepțiile checked

119. Observer Pattern si Singleton Pattern, din punct de vedere al clasificării Gang of Four, sunt:
- Observer este un design pattern behavioural, iar Singleton este structural
- Observer este un design pattern structural, iar Singleton este behavioural
- **Observer este un design pattern behavioural, iar Singleton este creational**
- ambele sunt structural

120. Care dintre urmatoarele afirmații sunt adevarate?

1. Metodele dintr-o interfață pot avea specificatorii de acces public sau default
2. String și Integer sunt clase imutabile (immutable)
3. Clasele abstracte nu pot fi instanțiate
4. Constructorii sunt folosiți pentru a inițializa un obiect nou creat

- 1, 2, 3
- 2, 4
- 3, 4
- **2, 3, 4**

Grile POO 2020

121. Cand apare NullPointerException?

- Cand ıncercam sa deschidem un fisier care nu exista
- Cand facem buffer overflow la un array
- **Cand ıncercam sa folosim un obiect null**
- Cand folosim o colectie care este instantiata si goala

122. Care este rezultatul execut, iei codului de mai jos?

```
String a = "hello";
String b = "world";
String c = b;
b = a + b;
String d = new String("hello");
System.out.println(c + " " + (d == a));
```

- **world false**
- helloworld true
- hello true
- helloworld false

123. Care varianta defineste cel mai bine legatura dintre interfete si clase?

- Nu pot exista relatii ıntre clase si interfete
- Nici clasele, nici interfetele nu precizeaza modul ın care un obiect executa o operatie
- **Interfetele precizeaza operatiile expuse de un obiect, ın timp ce clasele modul ın care acesta le executa**
- Atat clasele, cat si interfetele definesc modul ın care un obiect executa o operatie

124. Ce afirmatie despre Set si List este adevarata?

- In Set nu avem elemente duplicate, iar Set extinde interfata List
- In Set putem avea elemente duplicate, la fel ca ın List

- Nici in Set, nici ın List nu avem elemente duplicate
- **In Set nu avem elemente duplicate, iar ın List putem sa avem**

125. Fie:

```
class Device {
        public static void charge() {
                System.out.print("Charging device; ");
        }
}
class Phone extends Device {
        public static void charge() {
                System.out.print("Charging phone; ");
        }
}
```

Care este rezultatul executiei codului de mai jos?

```
Device phone = new Phone();
Device device = new Device();
phone.charge();
device.charge();
Device.charge();
```

- Charging device; Charging device; Charging phone;
- Charging phone; Charging phone; Charging device;
- **Charging device; Charging device; Charging device;**
- Charging phone; Charging device; Charging device;

126. Care afirmatii sunt adevarate despre clase interne statice?

1. Nu au acces la membri non-statici ai clasei exterioare
2. Este nevoie de o instanta a clasei externe pentru a o instantia
3. Trebuie sa mosteneasca clasa exterioara
4. Trebuie instantiata astfel: Outer out = new Outer(); Inner in = out.new Inner();

- 3
- **1**
- 2, 4
- 1, 2

127. In Java, prin compararea a doua obiecte cu ==, ce se compara?

- tipurile obiectelor
- marimile obiectelor
- **adresele obiectelor**
- valorile obiectelor

128. Fie un serviciu de tip Youtube, ın care avem utilizatori care se pot abona la canale, putand fi la curent cu noutatile canalelor la care ei sunt abonati (ıncarcare

videoclip nou, depasirea unui anumit numar de abonati etc). Ce design pattern se poate aplica aici?

- Visitor
- Factory
- **Observer**
- Command

129. Care dintre urmatoarele afirmatii sunt adevarate?

1) Clasele abstracte pot avea keyword-ul final
2) O clasa poate mosteni mai multe clase ın acelasi timp
3) O interfata poate mosteni mai multe interfete ın acelasi timp
4) Metodele si atributele de tip protected pot fi accesate doar din clasa-origine si din clasele derivate

- 1, 2, 3
- **3, 4**
- 2, 3, 4
- 1, 2, 3, 4

130. Daca Feline extinde clasa abstracta Animal si Lion extinde Feline, atunci care instantiere este corecta?

- Feline feline = new Animal();
- **Animal animal = new Feline();**
- Lion lion = new Animal();
- Lion lion = new Feline();

131. Ce ınseamna constructorul default?

- constructor fara parametri declarat de utilizator
- constructor fara implementare
- constructor fara modificatori de acces
- **constructor fara parametri adaugat de Java daca nici un constructor nu a fost declarat**

132. Ce design pattern putem folosi pentru a avea un obiect unic si vizibil la nivel global?

- Observer
- Visitor
- **Singleton**
- Factory

133. In ce colectie de tip Set se pastreaza ordinea insertiei elementelor?

- AbstractSet
- **LinkedHashSet**

- HashSet
- TreeSet

134. Ce colectie ar fi mai eficienta de folosit daca dorim sa stocam o secventa de elemente pe care sa o modificam rar dar pe care sa o accesam foarte des?
- **ArrayList**
- LinkedList
- niciuna din variante
- Vector

135. Ce rol are un copy constructor?
- de a copia datele si referinta unui obiect
- de a copia referinta unui obiect
- de a distruge referinta unui obiect
- **de a copia datele unui obiect**

136. Care este rezultatul executiei codului de mai jos?

```
Food food1 = new Food();
Food food2 = new Cake();
Cake cake = new Cake();
food1.cut();
food2.cut();
cake.cut();
Unde:
        class Cake extends Food {
                public void cut() {
                        System.out.print(" Cake ");
                }
        }
        class Food {
                public void cut() {
                        System.out.print(" Food ");
                }
        }
```

- Food Food Cake
- Food Food Food
- **Food Cake Cake**
- Cake Cake Cake

137. Identificati pe care linii avem suprascriere (overriding) si pe care avem supraıncarcare (overloading)

```
interface Coffee {
        void brew(); // 1
        void drink(); // 2
```

```
}
class Espresso implements Coffee {
        public void brew() { } // 3
        public void drink() { } // 4
        public void drink(Boolean withSugar) { } // 5
        public void drink(Boolean withMilk,
                Boolean withSugar) { } // 6
}
```

- Suprascriere: 5,6; Supraıncarcare: 3,4
- Suprascriere: 1,2,3; Supraıncarcare: 4,5,6
- Suprascriere: 3; Supraıncarcare: 4,5,6
- **Suprascriere: 3,4; Supraıncarcare: 5,6**

138. Ce se va afisa la rularea codului?

```
class Foo {
        public int doWork(int x) {
                try {
                        X++;
                        if (x < 3) {
                                throw new Exception();
                        }
                        return ++x;
                } catch (Exception e) {
                        return x++;
                } finally {
                        return ++x;
                }
        }
}
class Main {
        public static void main(String[] args) {
                Foo f = new Foo();
                System.out.println(f.doWork(1));
        }
}
```

- 3
- **4**
- 2
- 5

139. Ce se va afisa la rularea codului?

```
class Foo {
        public static void doWork() {
```

```
                System.out.print(Done! ");
        }
}
class Main {
        public static void main(String[] args) {
                Foo f = null;
                f.doWork();
                Foo.doWork();
        }
}
```

- **”Done! Done!”**
- La compilare va da o eroare la f.doWork();
- ”Done! ”
- La rulare programul va da NullPointerException

140. Ce reprezinta implementarea unei clase anonime?

- **implementarea unei interfete sau extinderea unei clase**
- implementarea de metode anonime
- implementarea unei interfete ın mod exclusiv
- crearea unei clase care are un constructor

Grile POO 2021

141. Având în vedere codul de mai jos, care dintre afirmațiile de mai jos sunt adevărate în ceea ce privește conceptul de imutabilitate aplicat în cazul clasei Student?

```
class Student {
        long id;
        List<Double> grades;
}
```

1) Ambele câmpuri să fie final
2) Să avem doar metode de tip getter pentru câmpuri
3) Să avem metode de tip getter și setter pentru câmpuri
4) În getter-ul pentru grades, trebuie să întoarcem o copie a obiectului grades
5) În getter-ul pentru grades, putem să întoarcem pur și simplu grades

- toate
- **1, 2, 4**
- 1, 2, 5
- 1, 3, 5

142. Care dintre afirmațiile de mai jos sunt adevărate?

1) Într-un Set putem avea duplicate
2) În LinkedHashMap se păstrează ordinea de inserție a elementelor

3) Interfața Map extinde interfața Collection
4) În TreeSet nu se păstrează ordinea de inserție a elementelor

- toate
- **2 și 4**
- 1, 2, 3
- 1, 3

143. Care dintre următoarele afirmații despre clase abstracte în Java sunt false:
1) Dacă derivăm o clasă abstractă și nu implementăm toate metodele abstracte, atunci clasă derivată trebuie de asemenea să fie abstractă
2) Clasele abstracte pot avea constructori
3) O clasă nu poate fi abstractă fără o metodă abstractă
4) O clasă abstractă poate moșteni de la mai multe clase abstracte

- 1, 2
- 1, 3
- **3, 4**
- 1, 3, 4

144. Care dintre următoarele afirmații despre interfețe și clase abstracte sunt adevărate?
1) O clasă poate moșteni în același timp mai multe clase abstracte
2) O interfață poate moșteni mai multe interfețe în același timp
3) O interfață poate moșteni o clasă abstractă care are doar metode abstracte
4) O clasă poate implementa mai multe interfețe în același timp

- 1, 2, 3
- **2, 4**
- 1, 2, 3, 4
- 2, 3, 4

145. Care dintre următoarele afirmații e corectă?
- Cuvântul final în fața unei metode arată faptul că acea metodă poate fi apelată doar în clasa respectivă
- Cuvântul final în fața unei clase înseamnă că nu poate fi instanțiată
- **Cuvântul final în fața unei variabile arată că acea variabilă poate fi asignată o singură dată**
- Cuvântul final în fața unei variabile arată că acea variabilă nu poate fi moștenită

146. Care dintre următoarele variante nu are legătură cu încapsularea?
- expunerea unei interfețe de lucru cu obiectul
- accesul la membri privați folosind metode de tip getter și setter
- construirea de obiecte complexe și ascunderea modului lor de funcționare

- **posibilitatea suprascrierii (overriding) metodelor**

147. Care este outputul următorului program?

```
public class Star {
        private int shininess;

        static void change(Star s, int newShine) {
                s = new Star();
                s.shininess = newShine;
        }

        public static void main() {
                Star star = new Star();
                System.out.println("Old shininess:" + star.shininess + "|");
                change(star, 10);
                System.out.println("New shininess:" + star.shininess);
        }
}
```

- Old shininess: 0 | New shininess: 10
- **Old shininess: 0 | New shininess: 0**
- Old shininess: null | New shininess: 0
- Old shininess: null | New shininess: 10

148. Care instanțiere este corectă?
- List<Integer> list = new ArrayList<Object>();
- ArrayList<Integer> list = new List<Integer>();
- **List<Integer> list = new ArrayList<Integer>();**
- ArrayList<Object> list = new ArrayList<Integer>();

149. Ce design pattern face ușoară separarea operațiilor de obiectele pe care se execută operațiile respective?
- Builder Pattern
- **Visitor Pattern**
- Factory Pattern
- Observer Pattern

150. Ce design pattern trebuie să folosim în caz că dorim ca un client să nu aibă rol în crearea unui entități după o preferință de a sa?
- Singleton Pattern
- **Factory Pattern**
- Builder Pattern

- Observer Pattern

151. Ce fel de Singleton este această secvență de cod?

```
public class Database {
        private static Database obj = null;

        private Database() {

        }

        public static Database getInstance() {
                if (obj == null) {
                        obj = new Database();
                }
                return obj;
        }
}
```

- Singleton cu inițializare eager
- **Singleton cu inițializare lazy**
- nu avem Singleton aici
- Singleton cu inițializare în blocuri statice

152. Ce metode pot fi accesate din clasa MyTest?

```
package test;
public class Test {
        void execute1() {}
        protected void execute2() {}
        private void execute3() {}
}
…
package mytest;
public class MyTest extends Test {
        public void execute2() {}
}
```

- execute1, execute2 din Test, execute2 din MyTest
- execute1, execute2 din Test, execute3, execute2 din MyTest
- **execute2 din Test, execute2 din MyTest**
- execute2 din MyTest

153. Ce reprezintă o metodă default în cadrul unei interfețe?

- o metodă abstractă
- **o metodă non-statică care este deja implementată într-o interfață**

- o metodă statică implementată într-o interfață
- nu putem avea metode default într-o interfață

154. Ce se afișează la rularea acestui cod?

```
class Document {}
class PdfDocument extends Document {}
class CsvDocument extends Document {}

class DocumentViewer {
        public void openDocument(Document document) {
                System.out.println("Opening document");
        }
        public void openDocument(PdfDocument document) {
                System.out.println("Opening pdf");
        }
        public void openDocument(CsvDocument document) {
                System.out.println("Opening csv");
        }
}

class AdobeReader extends DocumentViewer {
        public void openDocument(PdfDocument document) {
                System.out.println("Adobe is opening pdf");
        }
        public void openDocument(CsvDocument document) {
                System.out.println("Cannot open csv");
        }
}

class DocumentTest {
        public static void main(String[] argos) {
                DocumentViewer documentViewer = new AdobeReader();
                Document file1 = new PdfDocument();
                CsvDocument file2 = new CsvDocument();

                documentViewer.openDocument(file1);
                documentViewer.openDocument(file2);
        }
}
```

- **Opening document Cannot open csv**
- Adobe is opening pdf Cannot open csv

- Adobe is opening pdf Opening csv
- Opening document Opening csv

155. Ce se va afișa la execuția codului de mai jos?

```
public class Test {
    public static void main(String[] args) {
        A.B.write();
        A a = new A();
        A.write();
        A.B.write();
    }
}

class A {
    static boolean x = false;
    A() {
        x = false;
        B.x = false;
    }
    static class B {
        static boolean x = false;

        Public static void write() {
            x = !x;
            if (x) System.out.print(“A”);
            else System.out.print(“B”);
        }
    }

    public static void write() {
        x = !x;
        if (x) System.out.print(“C”);
        else System.out.print(“D”);
    }
}
```

- BCA
- **ACB**
- ADB
- BDA

156. Ce tip de clasă reprezintă următoarea implementare?

```
Predicate<Integer> check = x -> x % 2 == 0;
```

- clasă locală

- **clasă anonimă**
- clasă nested non-statică
- clasă nested statică

157. Ce va afișa programul?

```
public class Test {
        public static void main(String[] args) {
                Test obj = new Test();
                obj.first();
        }

        void first() {
                String a = “first”;
                String b = second(a);
                System.out.print(“:” + a + b);
        }

        String second(String a) {
                a = a + “luck”;
                System.out.print(a);
                return “luck”;
        }
}
```

- goodluck:goodluckluck
- goodluck:goodgoodluck
- **goodluck:goodluck**
- goodluck:luckgoodluck

158. Identificați pe care linii avem suprascriere (overriding) și pe care avem supraincărcare (overloading) corecte, fără erori de compilare:

```
public class Test {
        public static void main(String[] args) {
                AudioBook book = new AudioBook();
        }
}

class BookException extends Exception {}

interface Book {
        void markRead();
        void markToRead() throws BookException;
}
```

```
class AudioBook implements Book {
        public void markRead() {} //1
        public void markToRead() {} //2
        public void markRead(int rating) {} //3
        public void markRead(int rating, Date date) {} //4
}
```

- Suprascriere: 1, Supraincărcare: 3, 4
- **Suprascriere: 1, 2, Supraincărcare: 3,4**
- Suprascriere: 3, 4, Supraincărcare: 1, 2
- Suprascriere: 3, 4, Supraincărcare: 1

159. În ce zonă de memorie sunt stocate obiectele?

- pe stivă
- **pe heap**
- în memoria cache
- memoria externă

160. Pentru ce folosim map() pe colecții și pe arrays în Java?

- pentru a selecta elementele dintr-o colecție / dintr-un array care respectă o condiție reprezentată de funcția lambda dată că parametru
- **pentru a aplica o operație pe fiecare element din cadrul colecției sau al array-ului, operație reprezentată de funcția lambda dată că parametru**
- pentru a reduce elementele unei colecții / unui array la un singur element prin operația reprezentată de funcția lambda dată că parametru
- pentru a crea o nouă copie a colecției / al array-ului